

Vaccine Sentiment Analysis Using Sequential NLP Models

Group 18: Adossi Fred William, Honourgod Kilechukwu Levison, Parfait Christian Henry UHIRIWE, Serein Byiringiro Shima

Machine Learning Techniques I, Formative Assignment 2 · African Leadership University · October 2026

GitHub repository: <https://github.com/Adossi-design/vaccine-sentiment-sequential-models> · **Demo video:** [add link] · **Contribution tracker:** [add link]

Who built what: Adossi Fred William built the data investigation, the shared loading, cleaning, split and scoring code (src/data.py, src/preprocess.py, src/evaluate.py), and TF-IDF + Ridge and TF-IDF + SVR (notebook 01); Honourgod Kilechukwu Levison the Simple RNN (notebook 02); Parfait Christian Henry UHIRIWE the LSTM (notebook 03); and Serein Byiringiro Shima the GRU with its GloVe round (notebook 04), the shared training helpers (src/common.py) and the five-model comparison and error analysis (notebook 05, with contributions from Adossi Fred William, Honourgod Kilechukwu Levison and Parfait Christian Henry UHIRIWE).

1. Introduction

Vaccine hesitancy, the reluctance or refusal to vaccinate despite vaccines being available, was named one of ten threats to global health by the World Health Organization (2019). Decisions about vaccination are now shaped as much by what people read and share online as by what a doctor tells them, and a shift in mood on social media can show up before it changes vaccination rates. Crowdbreaks, the platform that supplied our labelled tweets, was built to track this: it collects public tweets about health topics and has crowd workers label their sentiment, so that researchers and health teams can watch opinion move over time (Müller & Salathé, 2019).

The tweets come from a challenge on Zindi, an African data science platform (Zindi, 2020). Each is labelled -1 (negative), 0 (neutral) or +1 (positive), and the task is to predict that label as a continuous score judged by root mean squared error (RMSE). The tweets are almost entirely in English, so this project says more about these methods on English social media text than about vaccine sentiment in Africa's many languages. The task is a sequence problem because meaning depends on word order: a negation such as “don't vaccinate” reverses the words after it, and sarcasm can invert what the words say. A model that only counts words cannot tell these cases apart, and counts of word pairs catch only a negation sitting right next to the word it changes.

This leads to the question this project investigates: how well do recurrent sequence models, specifically a Simple RNN, an LSTM and a GRU, predict vaccine sentiment in tweets compared with TF-IDF baselines, and what explains where they fail?

The short answer is that all three recurrent models beat always predicting the average label (RMSE 0.5769 to 0.5855 against 0.6459), but a tuned TF-IDF + Ridge model beats every one of them (0.5628), and starting the GRU from pretrained GloVe vectors narrows that gap without closing it (0.5700 over three seeds). Sections 2 and 3 review related work and the data, section 4 describes how each model

was built and tuned, section 5 compares the results, section 6 analyses the errors and limitations, and section 7 concludes.

2. Related Work

2.1 Vaccine sentiment on social media

The labels in our data come from Crowdbreaks, a platform that collects public tweets about health topics and asks crowd workers to label them (Müller & Salathé, 2019). The same platform supplied one of the two datasets used by Naseem et al. (2021). Their measles and vaccination tweets had a share of anti-vaccine posts (about 11%) close to ours (10.4%). Their best model, a bidirectional GRU with attention built on a language model pretrained on vaccine tweets, reached an F1 of 0.819 on that dataset, and models built on COVID-Twitter-BERT (Müller et al., 2023) were the strongest of their baselines. Zhang et al. (2020) found a similar pattern for tweets about HPV vaccines, where a fine-tuned BERT model beat SVM baselines and recurrent models with attention. These studies treat sentiment as three classes and report F1, so their scores cannot be compared directly with our RMSE. They do show that pretrained language models lead on this task. Our main models learn their word vectors from scratch, so our study asks a narrower question about what recurrent models can learn from 8,001 tweets alone; one extra GRU round tests whether GloVe vectors pretrained on 2 billion tweets (Pennington et al., 2014b) close part of that gap. For African languages, the AfriSenti benchmark offers more than 110,000 labelled tweets in 14 languages (Muhammad et al., 2023). Our tweets are almost all in English, so our results say little about vaccine sentiment in African languages.

2.2 Bag-of-words baselines

A sequential model is only worth its extra cost if it beats a model that ignores word order, so we included two baselines built on TF-IDF. TF-IDF weights each word by how often it appears in a tweet and how rare it is across all tweets (Salton & Buckley, 1988), which gives a sparse vector with no word order. On top of it we used Ridge regression, which shrinks the weights to cope with many correlated features (Hoerl & Kennard, 1970), and support vector regression, which ignores errors smaller than a margin called epsilon (Drucker et al., 1997). Such linear models are strong baselines for short sentiment text, especially with word pairs (Wang & Manning, 2012), so they set a fair target for the recurrent models.

2.3 Simple recurrent networks

A SimpleRNN layer reads a sequence directly: at every token it combines the current word's embedding with one hidden state carried from the previous token, following the Elman network (Elman, 1990). It has no gates, which is its known weakness: training passes the error back through every step, and with a plain recurrence the signal tends to shrink or grow at each one, so a link between an early word and the prediction becomes hard to learn in long sequences (Bengio et al., 1994; Pascanu et al., 2013). Our tweets are short (median 17 tokens), well inside the range where this is usually manageable, so the Simple RNN is our reference point for judging whether the LSTM and GRU's gates earn their extra complexity.

2.4 Long short-term memory

The LSTM adds a memory cell controlled by gates, which lets a network keep information over many steps without the gradient fading as quickly as in a plain recurrent network (Hochreiter & Schmidhuber, 1997). The forget gate lets it drop information that is no longer useful (Gers et al., 2000). LSTMs have worked well on tweets: a deep LSTM with attention was one of the top systems in the SemEval-2017 tweet sentiment task (Baziotis et al., 2017). Our tweets are short, but about a third of both anti-vaccine and pro-vaccine tweets contain a negation, and a memory cell should help carry a negation forward to the words it changes. We included the LSTM to test this, and two of its runs drew on further work: reading tweets in both directions (Schuster & Paliwal, 1997) and dropout (Srivastava et al., 2014).

2.5 Gated recurrent units and our choice of five approaches

The GRU (Cho et al., 2014) is a lighter gated unit. An update gate decides how much of the previous state to carry over and a reset gate decides how much of it feeds the new candidate state, and there is no separate memory cell, so it needs three blocks of recurrent weights where the LSTM needs four. Which gated unit works better depends on the data. Chung et al. (2014) found both clearly ahead of a plain recurrent network but could not rank them, and in a search of about 10,000 architectures the GRU beat a standard LSTM on every task except language modelling (Jozefowicz et al., 2015). Yin et al. (2017) found a GRU best for sentence sentiment, but its advantage over a CNN grew with sentence length, and on short sentences the two were comparable. Our tweets are short and we train on only 8,001 of them, so we included the GRU to test whether gating helps on short text with about a quarter fewer recurrent weights than the LSTM. Zhang et al. (2020) found that a recurrent model without pretrained embeddings did not beat a linear SVM on HPV-vaccine tweets, which is why two linear baselines complete the five approaches in Table 1.

We left two other families out on purpose. A CNN reads fixed windows of a few words, close to what word pairs already give the TF-IDF models, and Yin et al. (2017) found it level with a GRU on short sentences, so it would add little contrast. A fine-tuned transformer such as COVID-Twitter-BERT is the known leader on this task (Müller et al., 2023), but it brings knowledge from millions of outside tweets, which answers a different question from ours, and it needs GPU time that not every member had. The two baselines are also not two versions of one model. Ridge minimises squared error, the same loss that RMSE scores, while the SVR's epsilon-insensitive loss ignores small errors and grows only linearly for large ones, so it is pulled less by the noisy and disputed labels described in section 3. Section 5.1 shows the difference: Ridge leads on RMSE and MAE, while the SVR has the highest MAE of the five but ranks anti-vaccine tweets best (AUC 0.716 against 0.641).

Table 1. The five approaches and why we chose them.

Approach	Why we included it	Data finding behind it
TF-IDF + Ridge	A strong, cheap baseline that ignores most word order; shrinkage copes with tens of thousands of correlated word and word-pair features.	Specific hashtags and words carry much of the label (97.8% of #vaccinateyourkids tweets are positive), and negations are common, so word pairs such as

Approach	Why we included it	Data finding behind it
		“not safe” let a bag of words see a negation next to its word.
TF-IDF + SVR	A second linear baseline with a different loss: errors smaller than epsilon cost nothing and larger ones grow linearly rather than squared, so a few large errors weigh less.	Labels are noisy: 43 near-duplicate groups carry conflicting labels and all 239 tweets with 1/3 agreement are labelled negative.
Simple RNN	The simplest architecture that reads word order at all; included as a reference point for whether the LSTM and GRU's gates earn their extra complexity.	Tweets are short (median 17 tokens), inside the range where a plain RNN's vanishing-gradient weakness is usually still manageable.
LSTM	A memory cell and gates can carry a negation forward to the words it changes (Hochreiter & Schmidhuber, 1997).	About a third of both anti-vaccine (34.6%) and pro-vaccine (34.4%) tweets contain a negation.
GRU	Gating with fewer weights than the LSTM; GRU against LSTM depends on the data (Chung et al., 2014).	Only 8,001 training tweets, so a smaller recurrent layer leaves less room to overfit.

3. Dataset and Exploratory Analysis

The data is the training file of the Zindi challenge “To Vaccinate or Not to Vaccinate” (Zindi, 2020), labelled through Crowdbreaks (Müller & Salathé, 2019). Each row has a tweet ID, the text (usernames and links already replaced by placeholders), a label of -1, 0 or 1, and an agreement value, the share of the three annotators who chose that label. The test file has no labels, so only the training file is used. One tweet (RQMQ0L2A) contained an unquoted line break that split it over two lines; we repaired it rather than dropping it, leaving 10,000 labelled tweets with no missing values.

The labels are imbalanced (Figure 1): 49.1% neutral, 40.5% positive and only 10.4% negative, a mean label of 0.3016. The smallest class also has the weakest labels, since all 239 tweets with an agreement of 1/3 are negative while every other tweet has at least 2/3 (Table 2). We therefore expected the largest errors on negative tweets, and kept agreement out of the inputs because a new tweet would not have it.

Near-duplicates add noise. After lowercasing and removing mentions, links, “RT” and punctuation, 188 groups cover 655 tweets, the largest with 59 copies, and 43 groups carry conflicting labels. Predicting

each of these tweets by its group's mean label still gives an RMSE of 0.2435, so part of the error lies in the annotation, and a random split would leak copies from training into validation.

The tweets are short, with a median of 17 tokens and 99.9% at 32 tokens or fewer (Figure 2), and the median barely differs between labels (17 or 18). Length alone says little about sentiment, and with sequences this short we could not take for granted that the gates of the LSTM and GRU would help.

Several features that a standard cleaning would delete carry signal. Mentions appear in 54.3% of negative tweets against 38.2% of neutral and 39.5% of positive ones, links are most common in neutral tweets (48.6%), and the hashtag a tweet uses matters (97.8% of tweets with #vaccinateyourkids are positive, 90.5% of those with #cdcwhistleblower negative), so we kept all of these as tokens (section 5.2 tests this choice; the hashtags matter most). Other noise is rare, and code-switching is almost absent (Table 2).

Word order and context matter too. Negation marks that a tweet has an opinion, not which side it takes (Table 2), and topic words mislead: 27.2% of tweets that mention autism are negative, against 10.4% overall, although some of them defend vaccines, and most “MMR” tweets with no vaccine word are neutral because the abbreviation means something else there. We therefore kept negations and stop words, and checked these cases again in the error analysis.

Table 2. What the exploratory analysis found and what we did about it.

Finding	Evidence (notebook 01)	What we did
Imbalanced labels	-1: 10.4%, 0: 49.1%, +1: 40.5%	Stratified the split; class weights tried for the GRU
Disagreement sits in the negative class	All 239 tweets with 1/3 agreement are labelled -1 (23% of negatives)	Kept them; agreement never used as an input
Near-duplicates	188 groups covering 655 tweets; 43 groups with conflicting labels	Kept each group on one side of the split
Short tweets	Median 17 tokens; 99th percentile 28; longest 48	Recurrent models read up to 28 tokens
Long-tailed vocabulary	12,776 distinct training words; 7,112 seen once	Recurrent models kept words seen at least twice (5,664, so 8.2% of validation tokens are unknown); the TF-IDF models kept every word
Noisy text	Links 43.7%, mentions 40.4%, HTML entities 5.9% of tweets	Kept placeholders as tokens (removing them cost little, section 5.2); decoded entities

Finding	Evidence (notebook 01)	What we did
Negation shows opinion, not direction	In 34.6% of negative and 34.4% of positive tweets, 17.1% of neutral	Kept negation words; motivates reading word order
Misleading topic words	776 "MMR" tweets with no vaccine word, 748 of them neutral	Flagged as a likely error source
Almost no code-switching	6 tweets with non-Latin letters, 18 with accented letters	No language-specific processing

Figure 1. Label distribution and annotator agreement within each label (n = 10,000).

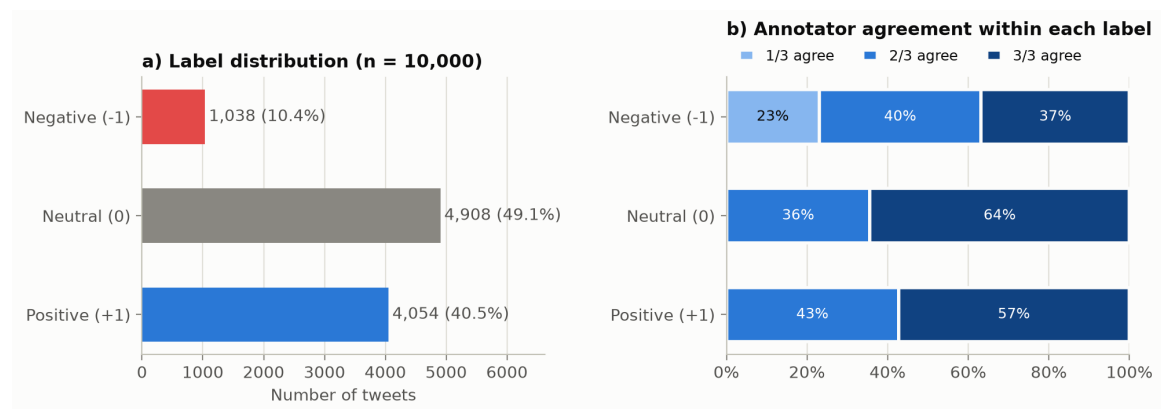
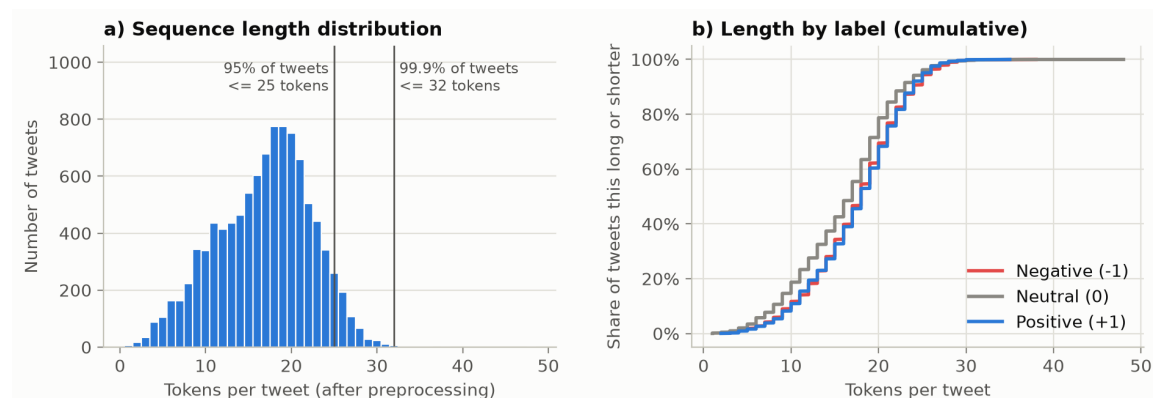


Figure 2. Tweet length in tokens after cleaning, overall and by label.



4. Methodology

4.1 Data preparation, split and evaluation

All five models share one cleaning function, one split and one way of scoring, so that differences in their results come from the models and not from the data.

The cleaning (`prepare_text` in `src/preprocess.py`) follows section 3. It decodes HTML codes, straightens curly quotes, shortens letters repeated three or more times to two and lowercases the text. It keeps the mention and link placeholders, hashtags, emojis and the marks `!`, `?` and `...` as tokens, keeps contractions such as “isn't” whole and removes the remaining punctuation. We did not remove stop words or stem, because stop-word lists include negations. The rules are fixed rather than fitted, so nothing is learned from the validation tweets, although the EDA that chose them covered all 10,000 tweets. To test these choices, notebook 05 retrained TF-IDF + Ridge and the GRU without stop words (scikit-learn's list, negations included, plus “n't” contractions; Pedregosa et al., 2011), without the mention and link placeholders, emojis and the marks `!`, `?` and `...`, without hashtags (the word included) and without all three (a standard cleaning).

We made one grouped, stratified 80/20 split with seed 42 and saved the tweet IDs: all copies of a near-duplicate fall on the same side, and the strata are label and agreement together. This gave 8,001 training and 1,999 validation tweets (206 negative, 982 neutral and 811 positive). Every vectorizer, tokenizer and vocabulary was fitted on the training tweets only.

Our main metric is RMSE, the competition metric named in the challenge's starter notebook (Zindi, 2020). It suits ordered labels and gives extra weight to large errors (Chai & Draxler, 2014), which fits a task where scoring an anti-vaccine tweet as pro-vaccine is the worst mistake, but one number mixes the average error with how unevenly the errors are spread (Willmott & Matsuura, 2005), so we report MAE next to it. Always predicting the training mean (0.3013) gives an RMSE of 0.6459 and an MAE of 0.5656. RMSE also says little about the rare anti-vaccine class that a monitoring tool most needs to find, so we add two secondary measures: per-class recall and macro F1 on predictions rounded to the nearest label, since macro F1 weights each class equally however rare it is (Sokolova & Lapalme, 2009), and the ROC AUC for separating anti-vaccine tweets from the rest by their predicted score, which needs no rounding threshold (Fawcett, 2006).

All code is Python. Data handling uses pandas (McKinney, 2010) and NumPy (Harris et al., 2020), the baselines and metrics use scikit-learn (Pedregosa et al., 2011), the recurrent models use TensorFlow (Abadi et al., 2016) through its Keras API (Chollet et al., 2015), and the figures use Matplotlib (Hunter, 2007).

4.2 TF-IDF + Ridge and TF-IDF + SVR

Both baselines are a scikit-learn pipeline (Pedregosa et al., 2011) of a TF-IDF vectorizer and a linear regressor, fitted on the training tweets only. The vectorizer splits the cleaned text on spaces, so the tokens stay exactly as the shared cleaning left them, and the SVR uses a linear kernel.

We tuned both one change at a time: each run changed one setting of the best run so far and kept the change only if it lowered the validation RMSE (Table B1 in Appendix B). Both started from the EDA (single words seen in at least two training tweets, raw counts and hashtags kept with their `#`; $\alpha = 1.0$ for Ridge, $C = 1.0$ and $\epsilon = 0.1$ for the SVR), and the order of the changes followed it too: word pairs first because negation is common, then log-scaled counts, then keeping every word to test whether rare words were worth their noise, then the regularisation and, for the SVR, ϵ , and finally removing the `#` from hashtags. This gave 7 runs for TF-IDF + Ridge and 8 for TF-IDF + SVR.

4.3 Shared recurrent setup and the Simple RNN

The three recurrent models share one input pipeline. Each cleaned tweet passes through a Keras TextVectorization layer adapted on the 8,001 training tweets only (Keras Team, n.d.-b). Following the long-tailed vocabulary (section 3), it keeps the 5,664 tokens seen at least twice, plus a padding id and an [UNK] id for every other word, and cuts or pads each tweet to 28 tokens, the 99th percentile of training length, which truncates 0.7% of training tweets. Padding is masked so the recurrent layer skips it. Each model has an embedding learned from scratch, one recurrent layer and one linear output unit, and is trained to minimise mean squared error with Adam at a learning rate of 0.001 (Kingma & Ba, 2015) and early stopping with a patience of 3 epochs (Prechelt, 1998).

The Simple RNN started from these settings with a 64-dimensional embedding, 64 units and 0.2 dropout, then changed one setting at a time in ten runs: sequence length, embedding size, units, dropout and learning rate, each at a lower and a higher value (Table B2). Retraining the baseline on two more seeds put the noise between two single runs at NOISE = 0.0029 (Reimers & Gurevych, 2017), and no change beat the baseline by more than that. The run kept, rnn-04, uses a 32-dimensional embedding, not because it scored meaningfully better but because it was the simplest setting within NOISE of the best, as in the one-standard-error rule (Hastie et al., 2009). It was trained on CPU.

4.4 LSTM

The LSTM used the same input and was trained on a Colab T4 GPU. The starting model (lstm-01) had a 64-dimensional embedding, 64 units and no dropout, so that its learning curve would show whether it overfits before any regularisation is added. It did: validation RMSE was lowest after the first epoch and rose afterwards. Two more seeds gave a mean of 0.5748 with a standard deviation of 0.0039, so a change had to beat that mean by more than 0.0039 to count.

We then changed one setting at a time (Table B3): dropout of 0.3 and a 32-dimensional embedding to test the overfitting, 128 units to test whether memory size limited the model, bidirectional reading to test whether later words such as a late negation help, weighting tweets by annotator agreement to test whether uncertain labels held it back, and a learning rate of 0.0003 because lstm-01 peaked in its first epoch. None of the six passed the threshold (Figure B1 in Appendix B), so we kept the starting settings. Three fresh seeds (7, 123 and 2026) then gave a mean of 0.5803 with a standard deviation of 0.0072, a wider spread than the first check suggested. This noise rule differs from the seed-42 comparison used for the Simple RNN and GRU, but the conclusion holds under either, since the largest gain over the seed-42 run was 0.0032 (lstm-05).

4.5 GRU

The GRU baseline (gru-01) used exactly the Simple RNN's baseline settings (embedding 64, 64 units, input dropout 0.2, batch size 64), so the first comparison between the two cells is like for like. Round 1 changed one setting at a time to a lower and a higher value, as for the Simple RNN (Table B4). Round 2 retrained the baseline with seeds 43 and 44, which put NOISE at 0.0072. Only dropout 0.4 (gru-09, 0.5769) beat the baseline by more than NOISE, and only just (by 0.0075), so there was nothing to combine. It became the final configuration and was checked on two fresh seeds, because a run chosen for scoring well tends to flatter itself. Recurrent dropout stayed at 0 so that the model can use the fast cuDNN kernel on a GPU (Keras Team, n.d.-a), although our runs used a CPU.

The error analysis then showed the GRU's main weakness: negative tweets received a mean prediction of 0.209, on the positive side of zero, and only 8 of the 206 fell below -0.5. Rounds 1 and 2 had only changed the network's size and training settings, so round 3 changed what the loss rewards, weighting each training tweet by its label (He & Garcia, 2009; Yang et al., 2021). Balanced weights give each label the same total weight (3.21 for a negative tweet against 0.68 for a neutral and 0.82 for a positive one), and square-root weights are a milder version.

Round 4 tested the gap named in section 2.1, that our word vectors come from 8,001 tweets alone. It kept the gru-09 settings and vocabulary but started the embedding from 100-dimensional GloVe vectors (Pennington et al., 2014a) trained on 2 billion tweets (Pennington et al., 2014b), either frozen or fine-tuned during training. GloVe has a vector for 5,043 of the 5,664 vocabulary words, covering 87.3% of validation tokens; the rest, which include contractions such as “don't” that GloVe splits, start from small random vectors. Every weighted and GloVe setting was trained with seeds 42, 43 and 44, so each run has a from-scratch, unweighted partner on the same seed. Table 3 sets the final GRU beside the other two recurrent models.

Table 3. Final settings of the three recurrent models, which share the input pipeline, loss, optimiser and early stopping of section 4.3 (lstm-final is lstm-01 retrained on seed 42).

Setting	Simple RNN	LSTM	GRU
Final run	rnn-04	lstm-final	gru-09
Embedding size	32	64	64
Recurrent units	64	64	64
Input dropout	0.2	0.0	0.4
Batch size	64	32	64
Parameters	187,585	395,713	387,649
Runs logged	15	13	27
Seeds for the final settings	3	6	3
Rule for a real improvement	Beat the seed-42 baseline by more than 0.0029	Beat the three-seed mean by more than 0.0039	Beat the seed-42 baseline by more than 0.0072
Trained on	CPU, TensorFlow 2.21	Colab T4 GPU, TensorFlow 2.20	CPU, TensorFlow 2.21

Because embedding size, dropout and batch size differ, a gap between two of these models mixes the cell type with each owner's tuning. The Simple RNN and GRU baselines share every setting, so notebook 05 also trained the LSTM on those settings with the same seeds (42, 43 and 44), through one code path in which only the recurrent layer differs.

5. Results and Discussion

5.1 Comparing the five approaches

Table 4 gives the validation results of the five models on the same 1,999 tweets. TF-IDF + Ridge had the lowest RMSE (0.5628), followed by TF-IDF + SVR (0.5740), the GRU (0.5769), the LSTM (0.5780) and the Simple RNN (0.5855), all well below the 0.6459 of always predicting the training mean. To see how much of this order could be chance, we resampled the validation tweets 2,000 times and scored every model on the same resamples (Koehn, 2004; Dror et al., 2018). The 95% intervals overlap (Figure 3), but most of their width comes from which tweets are drawn, which moves all five models together. On the same resample TF-IDF + Ridge beat every other model in at least 99.8% of cases, so its lead is small but not luck. The GRU and the LSTM beat the Simple RNN in 98.1% and 99.4% of resamples, and both gated cells also beat it when the three share the same settings (section 5.2). The GRU beat the LSTM in only 65.6% and TF-IDF + SVR beat the GRU in only 70.0%, so these three models, between 0.5740 and 0.5780, cannot be separated.

MAE gives a different order: TF-IDF + SVR had the second-lowest RMSE but the highest MAE (0.4626), and the LSTM the second-lowest MAE (0.4337) after TF-IDF + Ridge (0.4213), because RMSE rewards trading many small errors for fewer large ones (Chai & Draxler, 2014; Willmott & Matsuura, 2005). Clipping the predictions to the label range lowered every RMSE slightly, most for TF-IDF + Ridge (0.5628 to 0.5607), without changing the order.

RMSE also hides what happens to the negative class. Rounded to the nearest label (Figure 4), recall on negative tweets runs from 0.0% for the Simple RNN to 12.1% for TF-IDF + SVR, only 0 to 25 of the 206 negative tweets, and about three quarters of them are rounded to neutral. The ROC curves rank the predictions instead (Figure 5; Fawcett, 2006). TF-IDF + SVR ranked anti-vaccine tweets best, with an AUC of 0.716 against 0.641 for TF-IDF + Ridge and 0.550 for the Simple RNN, and at a 10% false-alarm rate it found 41.7% of them against 28.2% for Ridge; its AUC lead over Ridge held in every paired resample. Macro F1 separated the models less clearly, the SVR beating Ridge in only 88% of resamples. All five separated negative from positive tweets better (AUC 0.726 to 0.808) than negative from everything else, so the difficulty lies in telling negative tweets from neutral ones.

The model with the lowest RMSE is therefore not the best one for finding anti-vaccine tweets, and none of the five finds most of them. In our experiments a tuned bag of words and word pairs was not beaten by reading the words in order, which matches Wang and Manning (2012) on short sentiment text and Zhang et al. (2020) on HPV-vaccine tweets, where a recurrent model without pretrained embeddings did not beat a linear SVM.

Table 4. Validation results of the five approaches on the same 1,999 tweets. Lower RMSE and MAE are better; macro F1 uses predictions rounded to -1, 0 or 1; AUC is for separating anti-vaccine tweets from the rest. For the recurrent models, every column except the seed mean is the saved seed-42 run.

Approach	RMSE	95% interval	MAE	Mean $\pm$ sd over seeds	Macro F1	AUC
TF-IDF + Ridge	0.5628	0.542–0.583	0.4213	one run	0.502	0.641
TF-IDF + SVR	0.5740	0.555–0.593	0.4626	one run	0.513	0.716
GRU	0.5769	0.557–0.596	0.4436	0.5762 $\pm$ 0.0028 (3)	0.476	0.614
LSTM	0.5780	0.556–0.599	0.4337	0.5776 $\pm$ 0.0060 (6)	0.473	0.579
Simple RNN	0.5855	0.564–0.606	0.4458	0.5876 $\pm$ 0.0021 (3)	0.445	0.550
Always predict the mean	0.6459		0.5656			

Figure 3. Validation RMSE with 95% paired-bootstrap intervals; the dashed line is always predicting the mean.

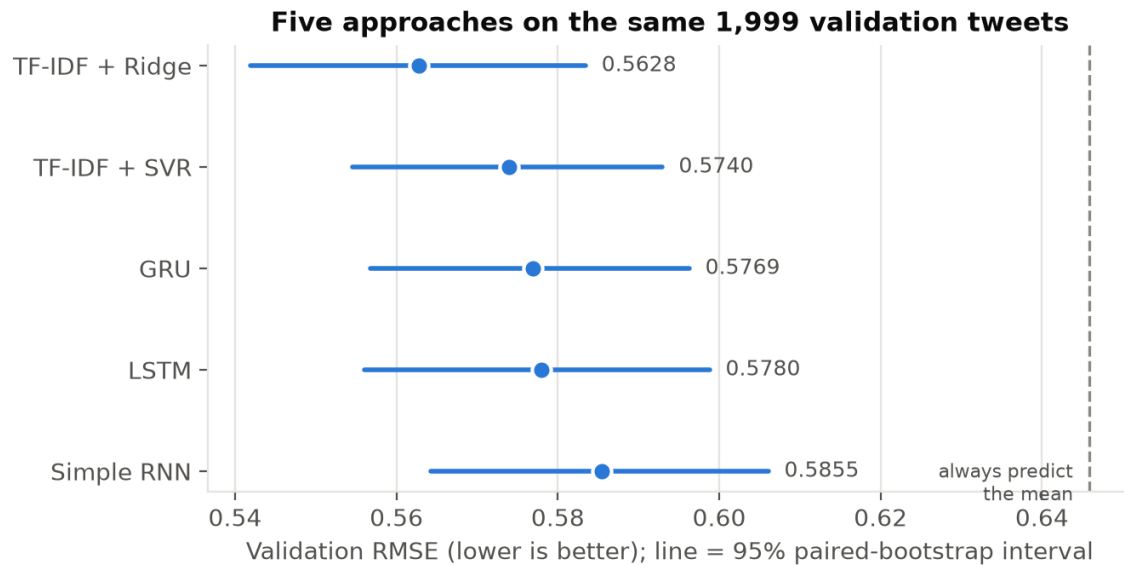


Figure 4. Confusion matrices after rounding each prediction to the nearest label.

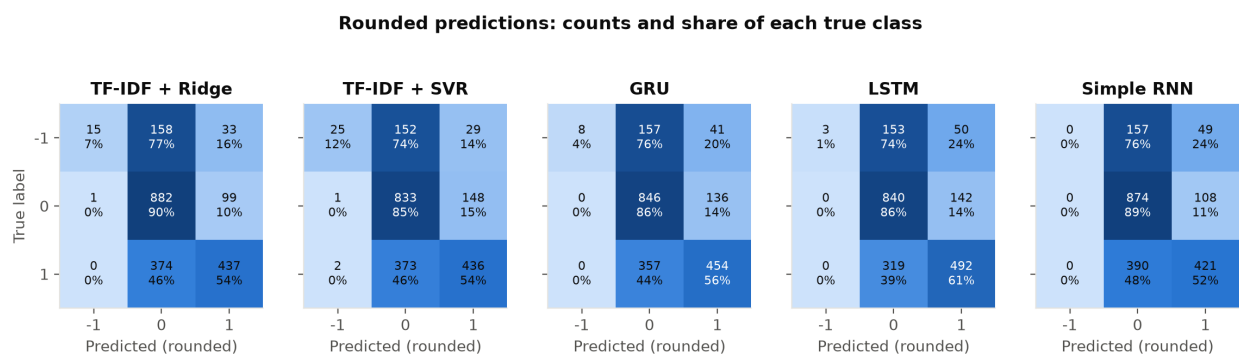
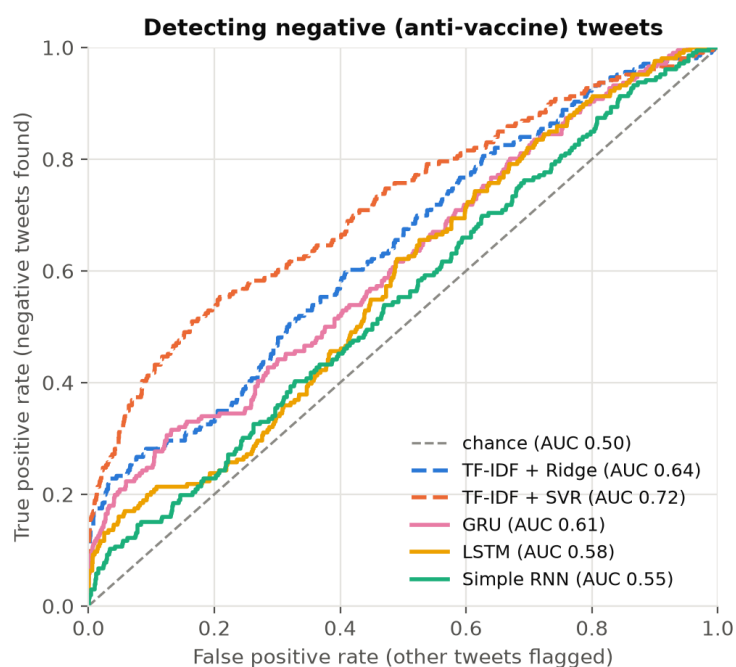


Figure 5. ROC curves for finding anti-vaccine tweets.



5.2 What each model's experiments showed

For both baselines the text representation mattered more than the model settings (Table B1). Word pairs gave TF-IDF + Ridge its largest gain (0.5791 to 0.5707), and keeping every word and word pair gave TF-IDF + SVR its largest (0.5917 to 0.5770). Stronger or weaker regularisation raised the RMSE of both models, log-scaled counts and removing the # changed it by about 0.001 or less, and a wider epsilon lowered the SVR's RMSE slightly while raising its MAE sharply.

Two of the Simple RNN's ten changes made it clearly worse, 128 units (+0.0122) and a learning rate of 0.003 (+0.0257), signs of overfitting or overshooting. Its validation RMSE was lowest after one or two epochs in every run, so these settings left it no room to improve (Figure 6a).

The LSTM also peaked within one or two epochs in every run, after which training RMSE kept falling while validation RMSE rose (Figure 6b). The two changes that raised its RMSE most (128 units and a 32-

dimensional embedding) were only 0.0016 and 0.0015 worse than the seed-42 run, so these settings were not what limited it.

For the GRU, the gain from dropout 0.4 shrinks from 0.0075 on seed 42 to about 0.004 between the three-seed means (0.5762 against 0.5800), although it was lower on every seed. Validation RMSE was lowest after the second epoch in 18 of the 21 runs of rounds 1 to 3 (Figure 6c). Class weighting traded the challenge metric for the anti-vaccine class (Table 5): balanced weights raised the AUC for anti-vaccine tweets to 0.730, level with TF-IDF + SVR, but also the RMSE, to 0.6447, no better than always predicting the mean, and macro F1 stayed at 0.46 because positive tweets were pushed down along with the negative ones.

Pretrained vectors helped only when fine-tuned (Table 5). Fine-tuned GloVe lowered the RMSE on every seed, from a three-seed mean of 0.5762 to 0.5700, and gave the largest macro-F1 gain of any GRU change (0.462 to 0.496), but the mean gain (0.0062) is below NOISE (one seed's gain, 0.0096, exceeds it), so gru-09 stayed the saved model. The gain came from neutral and positive tweets: the AUC for anti-vaccine tweets fell slightly (0.618 to 0.602), and frozen GloVe lowered it to 0.553 without changing the RMSE. General word vectors help the model read ordinary words, but stance on vaccines depends on context that co-occurrence statistics from general tweets do not carry, which fits the advantage of domain-specific pretraining reported by Müller et al. (2023) and Naseem et al. (2021).

On the shared settings (section 4.5) both gated cells beat the Simple RNN on every seed (three-seed means 0.5737 for the LSTM, 0.5800 for the GRU and 0.5886 for the Simple RNN; notebook 05, runs lfl-* in results/followup_experiments.csv), by more than NOISE on average, so the gain from gating does not come only from each owner's tuning. The LSTM also beat the GRU on every seed, but by 0.0064 on average, within the GRU's NOISE though not the LSTM's, and the tuned models rank them the other way, so the two gated cells cannot be ranked.

The cleaning check supports the choice made in section 3 (Table 6). Ridge is one deterministic fit per variant, so its changes are single measurements; only the GRU, trained on three seeds, can be judged against its NOISE (0.0072). A standard cleaning raised the RMSE of TF-IDF + Ridge by 0.0234 and the GRU's three-seed mean by 0.0197, the GRU being worse by more than NOISE on every seed. The hashtags, only 3.3% of the training tokens, are the largest single part of that cost: deleting them, the word included, cost Ridge 0.0101 and the GRU 0.0097, the GRU beyond NOISE on every seed. Removing the stop words, every negation included, cost Ridge 0.0045 and the GRU 0.0061, which for the GRU is below NOISE on average although worse on every seed and beyond NOISE on one (0.0081); removing the mentions, links, emojis and !, ? and ... cost Ridge 0.0039 and the GRU 0.0026, the GRU within NOISE on every seed. Removing all three cost Ridge more than its single costs added together (0.0234 against 0.0184), and only then did the GRU's AUC for anti-vaccine tweets fall close to chance (0.618 to 0.537).

Figure 6. Training and validation RMSE per epoch for the saved (a) Simple RNN, (b) LSTM and (c) GRU runs, retrained on CPU with seed 42 (notebook 05); the dashed line marks the epoch kept by early

stopping. The LSTM retrain reproduces the RMSE logged for lstm-final on a Colab T4 GPU (0.5780) to six decimals.

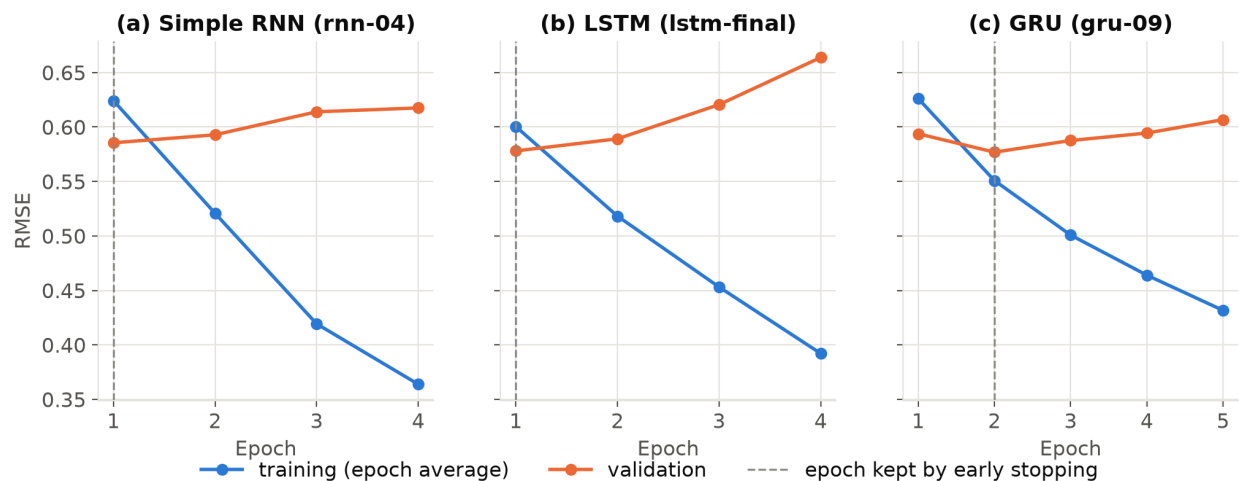


Table 5. GRU variants on the gru-09 settings, mean of three seeds (42, 43 and 44).

Variant	RMSE	Anti-vaccine tweets rounded to -1	Macro F1	AUC
Learned from scratch, no weighting	0.5762	5.3%	0.462	0.618
Square-root weights	0.5967	13.1%	0.462	0.684
Balanced weights	0.6447	24.9%	0.463	0.730
GloVe, frozen	0.5758	3.7%	0.469	0.553
GloVe, fine-tuned	0.5700	4.7%	0.496	0.602

Table 6. TF-IDF + Ridge (ridge-07, one fit) and the GRU (gru-09, mean of three seeds) retrained on reduced versions of the shared cleaning (notebook 05, runs clean-*); AUC is for anti-vaccine tweets.

Cleaning	Training tokens removed	Ridge RMSE	Ridge AUC	GRU RMSE	GRU AUC
Shared cleaning (used)	0%	0.5628	0.641	0.5762	0.618
Without stop words	35.8%	0.5673	0.617	0.5823	0.597
Without mentions, links, emojis, !, ? and ...	9.6%	0.5666	0.637	0.5788	0.615
Without hashtags (word included)	3.3%	0.5728	0.631	0.5859	0.600
Standard cleaning (all three removed)	48.6%	0.5862	0.597	0.5959	0.537

5.3 Where the models differ

Anti-vaccine tweets are the hardest group for all five models by a wide margin (Figure 7), with RMSE between 1.17 and 1.32 on that class against 0.30 to 0.35 on neutral tweets. Within that shared pattern the families split: the two TF-IDF baselines do best on anti-vaccine tweets, while the recurrent models do comparatively better on pro-vaccine tweets, the LSTM reaching an RMSE of 0.500 there. The linear models seem to pick up phrases that reliably mark anti-vaccine content, while the recurrent models do not turn their reading of order into an advantage on the class where order should matter most.

The negation result makes this concrete. Pooled over all tweets, a negation adds a similar 0.062 to 0.069 RMSE for every model, but tweets with a negation are more often pro-vaccine (55% against 36% without one), so the families only separate within each true label. On the 60 anti-vaccine tweets with a negation, the TF-IDF models do clearly better (SVR 1.20, Ridge 1.25) than the recurrent models (GRU 1.33, LSTM 1.41, Simple RNN 1.44), and the negation adds 0.05 to 0.07 to the TF-IDF error against 0.10 to 0.17 to the recurrent error. On the 280 pro-vaccine tweets with a negation the order flips (LSTM 0.44, Simple RNN 0.46, GRU 0.49, Ridge 0.51, SVR 0.55). The recurrent models seem to have learned a negation as a pro-vaccine cue, right on average in the training data but wrong for tweets telling people not to vaccinate. The 60 negated anti-vaccine tweets are a small group, so this is a pattern in the errors, not a direct test of what the networks compute.

The same holds for tweet length: on the longest third of tweets (21 to 36 tokens), where reading order should help most, the recurrent models still trail Ridge (0.61 to 0.62 against 0.60). This is unlike Yin et al. (2017), who saw a GRU's advantage over a CNN grow on longer sentences. No validation tweet is longer than 36 tokens (48 in the whole dataset), and the recurrent models read at most the first 28, so there is little long-range order for a recurrent model to exploit.

Figure 7. Validation RMSE by tweet group for each model; bold marks the best model in each row.

	TF-IDF + Ridge	TF-IDF + SVR	GRU	LSTM	Simple RNN
true label: negative (-1) (n=206)	1.20	1.17	1.26	1.31	1.32
true label: neutral (0) (n=982)	0.30	0.35	0.33	0.33	0.31
true label: positive (1) (n=811)	0.55	0.56	0.53	0.50	0.54
negative tweets by agreement: 1/3 (n=47)	1.29	1.30	1.35	1.39	1.38
negative tweets by agreement: 2/3 (n=83)	1.22	1.19	1.26	1.31	1.31
negative tweets by agreement: 3/3 (n=76)	1.12	1.04	1.20	1.27	1.31
negation: contains a negation (n=512)	0.61	0.62	0.63	0.63	0.64
negation: no negation (n=1487)	0.55	0.56	0.56	0.56	0.57
tokens after cleaning: 2 to 15 (n=724)	0.55	0.56	0.55	0.55	0.56
tokens after cleaning: 16 to 20 (n=729)	0.55	0.56	0.57	0.58	0.58
tokens after cleaning: 21 to 36 (n=546)	0.60	0.61	0.62	0.61	0.62
unknown words: none (n=691)	0.54	0.55	0.55	0.56	0.57
unknown words: up to 10% (n=655)	0.59	0.60	0.61	0.61	0.62
unknown words: over 10% (n=653)	0.56	0.57	0.57	0.57	0.56

6. Error Analysis and Limitations

6.1 Errors every model shares

Many of the hardest tweets are hard for every model. In our validation set, 123 tweets are badly wrong for all five models, meaning the prediction is off by more than one class step. 122 of them are anti-vaccine tweets, which is 59% of the 206 anti-vaccine tweets in the validation set (Figure 8). The models' errors are also highly correlated (0.91 to 0.99), so combining them does not help: averaging all five gives an RMSE of 0.5638, no better than TF-IDF + Ridge alone (0.5628).

Reading the shared hard cases suggests four patterns (Table 7): tweets that report anti-vaccine news instead of arguing a position (A3W81QVQ), tweets that negate a phrase that would otherwise sound pro-vaccine (5KDOWBMS), sarcasm (AP8D8KXB), and tweets whose stance rests on a claim, such as a joking link between childhood vaccines and autism (XRTV4HU8), which a model can only read as anti-vaccine with outside knowledge. Table 7 also gives each model's score: 47 of the 50 round to +1, the opposite of the label. On the three negation examples Ridge is further from -1 than every recurrent model, so these hand-picked cases run against the TF-IDF models' average advantage on negated anti-vaccine tweets (section 5.3).

Annotator disagreement explains part of these errors: 37 of the 47 validation tweets where only one annotator chose the recorded label are among the shared errors, and 30% of the shared hard cases have this low agreement, against 0.5% of the other tweets. It is not the whole story. Error falls as agreement

risers for every model, yet negative tweets on which all three annotators agreed still have an RMSE above 1 for every model, so many of the worst errors are on tweets whose labels are not in doubt.

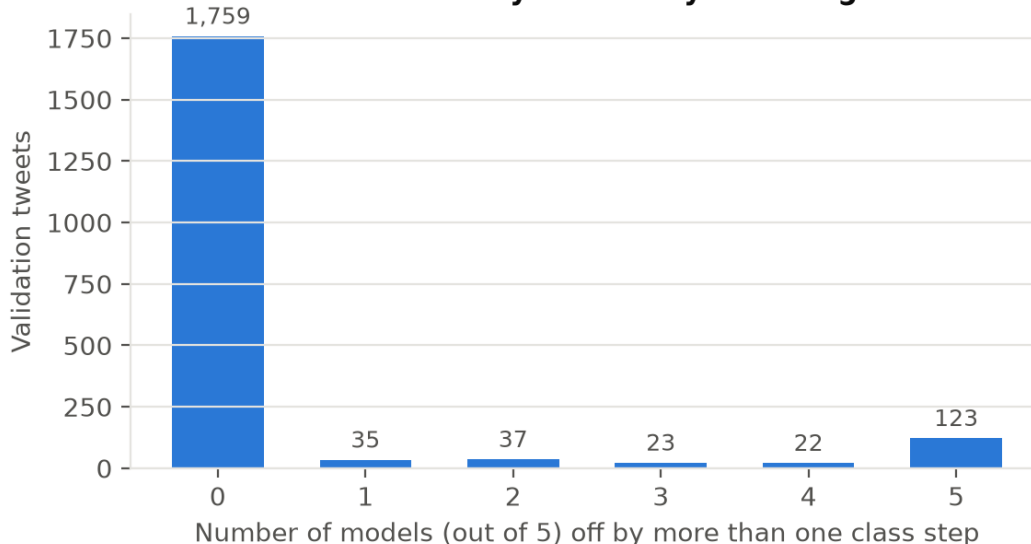
Code-switching and spelling variation are unlikely causes, since only 3 validation tweets contain accented letters and 21 elongated spellings. These patterns come from reading the hardest cases rather than from a full count, so they describe the kinds of tweets the models miss, not how often each kind occurs.

Table 7. Patterns among the tweets that all five models get badly wrong (122 of the 123 are labelled anti-vaccine, as are all the examples listed); after each ID, its annotator agreement and each model's predicted score (true label -1).

Pattern	Example tweet IDs (agreement) and scores: Ridge, SVR, Simple RNN, LSTM, GRU	Why the models miss it
Reporting anti-vaccine news rather than arguing	A3W81QVQ (2/3) 1.14, 0.94, 0.85, 0.81, 0.80 8FNX4C5D (1/3) 0.61, 0.48, 0.65, 0.79, 0.72 S1ZZQQHD (2/3) 0.75, 0.82, 0.57, 0.59, 0.58	The stance is reported rather than expressed, so the wording looks like neutral news, and whether such tweets are anti-vaccine at all is debatable.
Negating a pro-vaccine phrase	5KDOWBMS (3/3) 0.84, 0.56, 0.81, 0.81, 0.74 M4IVFSMS (3/3) 1.22, 1.09, 0.60, 0.64, 0.57 QK5Y1ZWX (2/3) 0.99, 0.88, 0.76, 0.83, 0.79	The tweet uses the same words as a pro-vaccine appeal, and only a negation reverses them, so models that lean on vaccine words read it as pro-vaccine.
Sarcasm	AP8D8KXB (1/3) 1.00, 0.90, 0.77, 0.90, 0.84 DVMK1WEI (2/3) 0.79, 0.81, 0.62, 0.90, 0.74	The literal words support vaccination and only the intended meaning opposes it, which word order alone does not reveal.
Stance carried by a claim or complaint (for example, vaccines and autism)	XRTV4HU8 (3/3) 0.43, 0.38, 0.85, 0.96, 0.83 4Y75EP2E (2/3) 0.71, 0.61, 0.83, 0.92, 0.76	The words are not negative in themselves, and seeing the stance requires knowing that a claim such as vaccines causing autism is an anti-vaccine argument.

Figure 8. Validation tweets by how many of the five models are off by more than one class step.

Most errors are shared: tweets by how many models get them badly wrong



6.2 Limitations

Data. Only 10.4% of the tweets are anti-vaccine, and every tweet whose three annotators all disagreed is labelled -1, so part of that class records disagreement rather than a clear stance. The tweets are almost all English.

Framing. Three ordered classes are treated as a score, so the squared-error loss pulls uncertain tweets towards the middle, which is why most anti-vaccine tweets are predicted neutral.

Models. Each recurrent model has one layer, the main runs learn their word vectors from 8,001 tweets, and only the GRU was tried with pretrained vectors (one round, two variants). Each owner tuned different settings under a different rule (Table 3), so gaps between the saved recurrent models mix architecture with tuning, which section 5.2 separates only on untuned settings.

Evaluation. There is one split and no test set. The validation set also chose the stopping epochs and settings, so every score is slightly optimistic, and the recurrent models were checked on only three to six seeds. Near-duplicate tweets lower every RMSE by about 0.01, without changing the ranking.

Preprocessing. The shared cleaning drops a plain three-dot ellipsis and turns “<3” into “3”, gaps found after training. The affected tweets show no extra error, so the models were not retrained. The cleaning check used one stop-word list, removed mentions, links, emojis and marks as one group, and kept settings tuned on our cleaning.

Environment. The LSTM was trained on a Colab GPU with TensorFlow 2.20 and the other models on CPU with TensorFlow 2.21; a CPU rerun matched the LSTM's logged RMSE exactly, but other setups may not.

6.3 Responsible AI

The class our models miss most is the one public-health teams care about most, so a monitoring tool built on them would under-count anti-vaccine content and could give false reassurance. The labels are

the judgement of three crowd workers, whose own views may shape them, and their disagreement concentrates in the anti-vaccine class, so the models learn from the least reliable labels where accuracy matters most. The data also over-represents some voices. Twitter users are not a sample of any population, and several near-duplicate groups in the training split hold 15 or more copies of one tweet, so a few accounts or campaigns can shape what a model learns a stance sounds like. A model that learns such wording as a stance cue may also misjudge people who write in other varieties of English, including African varieties, or who code-switch, which our nearly all-English data cannot test. The tweets are public posts by real people, so we never publish their text, and the models should not be used to identify or target individual users. Any real use should report only aggregate trends, keep a human reviewer in the loop, treat the alert threshold as an explicit policy choice made from the ROC trade-off (Figure 5), and be tested on data from the community where it would be used.

7. Conclusion and Future Work

Recurrent sequence models address this problem only moderately well. All three clearly beat always predicting the mean label, and the gated GRU and LSTM beat the Simple RNN even on identical settings, but none of them beat a tuned TF-IDF + Ridge model, which had the lowest RMSE of the five, and TF-IDF + SVR ranked anti-vaccine tweets best. On tweets this short, counts of words and word pairs capture most of the local order that matters, so reading the whole sequence added less than studies of longer text would suggest (Yin et al., 2017). Fine-tuned GloVe vectors brought the GRU to 0.5700, lower on every seed but still behind TF-IDF + Ridge, and they did not help it find anti-vaccine tweets. The shared cleaning also paid off: a standard cleaning would have raised the RMSE of TF-IDF + Ridge by 0.0234 and of the GRU by 0.0197, and deleting the hashtags, only 3.3% of the tokens, was the largest single part of that cost.

The errors point mainly at the data. The hardest tweets are the same for all five models: anti-vaccine tweets with disputed labels, stance that is reported rather than argued, sarcasm, and claims whose stance needs outside knowledge. Negation cuts both ways for the recurrent models, helping on pro-vaccine tweets and hurting on anti-vaccine ones. Part of the problem is the objective: under squared error the cheapest prediction for an uncertain tweet is near the mean, and weighting the loss let the GRU rank anti-vaccine tweets about as well as the SVR, but only at a large cost in RMSE.

These conclusions rest on one validation split with no test set, on recurrent models tuned under different rules, and on almost entirely English tweets, so the close rankings may change on new data and the results may not hold for African-language tweets. Five directions follow from these results. Disputed labels should be handled explicitly, for example by reporting results by agreement level or modelling disagreement as its own outcome. Models should be trained for the anti-vaccine class with a classification loss judged by macro F1 or AUC. Outside knowledge should come from a domain-adapted transformer such as COVID-Twitter-BERT (Müller et al., 2023), since general GloVe vectors improved the GRU only slightly and not on the anti-vaccine class. Predictions should be clipped to the label range, a small gain for every model at no cost. Finally, repeated or cross-validated splits would settle close rankings such as GRU against LSTM.

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., . . . Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (pp. 265–283). USENIX Association. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- Baziotis, C., Pelekis, N., & Doukeridis, C. (2017). DataStories at SemEval-2017 Task 4: Deep LSTM with attention for message-level and topic-based sentiment analysis. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)* (pp. 747–754). Association for Computational Linguistics. <https://doi.org/10.18653/v1/S17-2126>
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. <https://doi.org/10.1109/72.279181>
- Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature. *Geoscientific Model Development*, 7(3), 1247–1250. <https://doi.org/10.5194/gmd-7-1247-2014>
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734). Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1179>
- Chollet, F., & others. (2015). Keras [Computer software]. GitHub. <https://github.com/keras-team/keras>
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical evaluation of gated recurrent neural networks on sequence modeling* (arXiv:1412.3555). arXiv. <https://doi.org/10.48550/arXiv.1412.3555>
- Dror, R., Baumer, G., Shlomov, S., & Reichart, R. (2018). The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 1383–1392). Association for Computational Linguistics. <https://doi.org/10.18653/v1/P18-1128>
- Drucker, H., Burges, C. J. C., Kaufman, L., Smola, A., & Vapnik, V. (1997). Support vector regression machines. In M. C. Mozer, M. I. Jordan, & T. Petsche (Eds.), *Advances in Neural Information Processing Systems 9* (pp. 155–161). MIT Press.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211. https://doi.org/10.1207/s15516709cog1402_1
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10), 2451–2471. <https://doi.org/10.1162/089976600300015015>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A.,

- Fernández del Río, J., Wiebe, M., Peterson, P., . . . Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. <https://doi.org/10.1080/00401706.1970.10488634>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Jozefowicz, R., Zaremba, W., & Sutskever, I. (2015). An empirical exploration of recurrent network architectures. In *Proceedings of the 32nd International Conference on Machine Learning* (Vol. 37, pp. 2342–2350). PMLR. <https://proceedings.mlr.press/v37/jozefowicz15.html>
- Keras Team. (n.d.-a). GRU layer. In *Keras 3 API documentation*. Retrieved September 26, 2026, from https://keras.io/api/layers/recurrent_layers/gru/
- Keras Team. (n.d.-b). TextVectorization layer. In *Keras 3 API documentation*. Retrieved September 26, 2026, from https://keras.io/api/layers/preprocessing_layers/text/text_vectorization/
- Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization* [Paper presentation]. 3rd International Conference on Learning Representations (ICLR 2015), San Diego, CA, United States. <https://doi.org/10.48550/arXiv.1412.6980>
- Koehn, P. (2004). Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing* (pp. 388–395). Association for Computational Linguistics. <https://aclanthology.org/W04-3250/>
- McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majors-92bf1922-00a>
- Muhammad, S. H., Abdulmumin, I., Ayele, A. A., Ousidhoum, N., Adelani, D. I., Yimam, S. M., Ahmad, I. S., Beloucif, M., Mohammad, S. M., Ruder, S., Hourrane, O., Brazdil, P., Jorge, A., Ali, F. D. M. A., David, D., Osei, S., Shehu Bello, B., Ibrahim, F., Gwadabe, T., . . . Arthur, S. (2023). AfriSenti: A Twitter sentiment analysis benchmark for African languages. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 13968–13981). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.emnlp-main.862>
- Müller, M. M., & Salathé, M. (2019). Crowdbreaks: Tracking health trends using public social media data and crowdsourcing. *Frontiers in Public Health*, 7, Article 81. <https://doi.org/10.3389/fpubh.2019.00081>

- Müller, M. M., Salathé, M., & Kummervold, P. E. (2023). COVID-Twitter-BERT: A natural language processing model to analyse COVID-19 content on Twitter. *Frontiers in Artificial Intelligence*, 6, Article 1023281. <https://doi.org/10.3389/frai.2023.1023281>
- Naseem, U., Khushi, M., Kim, J., & Dunn, A. G. (2021). Classifying vaccine sentiment tweets by modelling domain-specific representation and commonsense knowledge into context-aware attentive GRU. In *2021 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE. <https://doi.org/10.1109/IJCNN52387.2021.9533454>
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning* (Vol. 28, No. 3, pp. 1310–1318). PMLR. <https://proceedings.mlr.press/v28/pascanu13.html>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Pennington, J., Socher, R., & Manning, C. D. (2014a). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1162>
- Pennington, J., Socher, R., & Manning, C. D. (2014b). *GloVe: Global vectors for word representation* (glove.twitter.27B) [Data set]. Stanford NLP Group. <https://nlp.stanford.edu/projects/glove/>
- Prechelt, L. (1998). Early stopping, but when? In G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (Lecture Notes in Computer Science, Vol. 1524, pp. 55–69). Springer. https://doi.org/10.1007/3-540-49430-8_3
- Reimers, N., & Gurevych, I. (2017). Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing* (pp. 338–348). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D17-1035>
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523. [https://doi.org/10.1016/0306-4573\(88\)90021-0](https://doi.org/10.1016/0306-4573(88)90021-0)
- Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), 2673–2681. <https://doi.org/10.1109/78.650093>
- Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958. <https://jmlr.org/papers/v15/srivastava14a.html>
- Wang, S., & Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics*

(Volume 2: Short Papers) (pp. 90–94). Association for Computational Linguistics.
<https://aclanthology.org/P12-2018/>

Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research*, 30(1), 79–82.
<https://doi.org/10.3354/cr030079>

World Health Organization. (2019). *Ten threats to global health in 2019*. <https://www.who.int/news-room/spotlight/ten-threats-to-global-health-in-2019>

Yang, Y., Zha, K., Chen, Y.-C., Wang, H., & Katabi, D. (2021). Delving into deep imbalanced regression. In *Proceedings of the 38th International Conference on Machine Learning* (Vol. 139, pp. 11842–11851). PMLR. <https://proceedings.mlr.press/v139/yang21m.html>

Yin, W., Kann, K., Yu, M., & Schütze, H. (2017). *Comparative study of CNN and RNN for natural language processing* (arXiv:1702.01923). arXiv. <https://doi.org/10.48550/arXiv.1702.01923>

Zhang, L., Fan, H., Peng, C., Rao, G., & Cong, Q. (2020). Sentiment analysis methods for HPV vaccines related tweets based on transfer learning. *Healthcare*, 8(3), Article 307.
<https://doi.org/10.3390/healthcare8030307>

Zindi. (2020). *To vaccinate or not to vaccinate: It's not a question* [Data set].
<https://zindi.africa/competitions/to-vaccinate-or-not-to-vaccinate>

Appendix A. Use of AI tools

Serein Byiringiro Shima used Claude (Anthropic) to check the notebooks and this report against the brief and against our results, and to help assemble and edit the report. [Adossi Fred William, Honourgod Kilechukwu Levison and Parfait Christian Henry UHIRIWE: add the AI tools you used and what for, or state that you used none.] Every number in this report comes from our own notebooks and experiments, and each member checked and can explain their own sections.

Appendix B. Tuning runs

Table B1. The 15 TF-IDF baseline runs on the shared validation set; each run changes one setting of the best run before it.

Run	Setting tested	Features	RMSE	MAE	Kept
ridge-01	starting settings (single words, words in at least 2 tweets, alpha 1.0)	5,569	0.5791	0.4389	start
ridge-02	add word pairs	19,187	0.5707	0.4313	yes
ridge-03	log-scaled term counts	19,187	0.5705	0.4308	yes
ridge-04	keep every word and word pair	77,644	0.5637	0.4226	yes
ridge-05	alpha 0.3	77,644	0.5757	0.4282	no

Run	Setting tested	Features	RMSE	MAE	Kept
ridge-06	alpha 3.0	77,644	0.5670	0.4337	no
ridge-07	remove the # from hashtags	76,458	0.5628	0.4213	yes (final)
svr-01	starting settings (single words, words in at least 2 tweets, C 1.0, epsilon 0.1)	5,569	0.5991	0.4395	start
svr-02	add word pairs	19,187	0.5924	0.4356	yes
svr-03	log-scaled term counts	19,187	0.5917	0.4352	yes
svr-04	keep every word and word pair	77,644	0.5770	0.4264	yes
svr-05	C 0.3	77,644	0.5824	0.4158	no
svr-06	C 3.0	77,644	0.5883	0.4404	no
svr-07	epsilon 0.3	77,644	0.5751	0.4634	yes
svr-08	remove the # from hashtags	76,458	0.5740	0.4626	yes (final)

Figure B1. Validation RMSE of the six single-change LSTM runs against the three-seed mean of the starting model (section 4.4): no change lowers the RMSE below the band of seed noise.

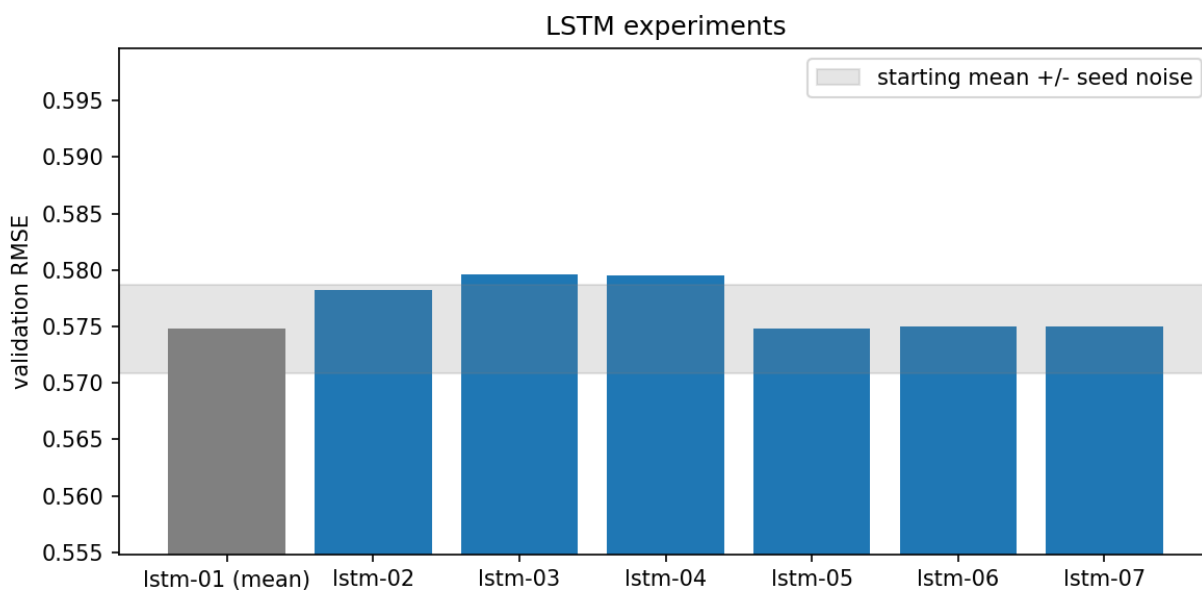


Table B2. The 15 Simple RNN runs (no run 14: nothing to combine). Round-1 runs change one setting of the starting settings; NOISE = 0.0029.

Run	Setting tested	Seed	RMSE	Best epoch	Result
rnn-01	starting settings (embedding 64, 64 units, dropout 0.2)	42	0.5866	1	start
rnn-02	sequence length 24	42	0.5880	1	no gain
rnn-03	sequence length 48	42	0.5871	1	no gain
rnn-04	embedding 32	42	0.5855	1	kept (final)
rnn-05	embedding 128	42	0.5897	1	no gain
rnn-06	32 units	42	0.5920	1	no gain
rnn-07	128 units	42	0.5988	2	no gain
rnn-08	dropout 0	42	0.5934	1	no gain
rnn-09	dropout 0.4	42	0.5876	1	no gain
rnn-10	learning rate 0.0003	42	0.5958	2	no gain
rnn-11	learning rate 0.003	42	0.6123	2	no gain
rnn-12	starting settings, new seed	43	0.5888	1	noise check
rnn-13	starting settings, new seed	44	0.5906	1	noise check
rnn-15	final settings (rnn-04), new seed	43	0.5878	2	seed check
rnn-16	final settings (rnn-04), new seed	44	0.5897	1	seed check

Table B3. The 13 LSTM runs. A change had to beat the three-seed starting mean (0.5748) by more than 0.0039.

Run	Setting tested	Seed	RMSE	Best epoch	Result
lstm-01	starting settings (embedding 64, 64 units, no dropout)	42	0.5780	1	start

Run	Setting tested	Seed	RMSE	Best epoch	Result
lstm-01-seed43	starting settings, new seed	43	0.5704	1	noise check
lstm-01-seed44	starting settings, new seed	44	0.5759	1	noise check
lstm-02	dropout 0.3	42	0.5782	1	no gain
lstm-03	128 units	42	0.5796	1	no gain
lstm-04	embedding 32	42	0.5795	1	no gain
lstm-05	bidirectional	42	0.5748	1	no gain
lstm-06	loss weighted by annotator agreement	42	0.5750	1	no gain
lstm-07	learning rate 0.0003	42	0.5750	2	no gain
lstm-final	final settings (= lstm-01), retrained	42	0.5780	1	kept (final)
lstm-final-seed7	final settings, new seed	7	0.5796	1	seed check
lstm-final-seed123	final settings, new seed	123	0.5878	2	seed check
lstm-final-seed2026	final settings, new seed	2026	0.5735	1	seed check

Table B4. The 27 GRU runs (rounds 1 to 4; no run 14: nothing to combine). Round-1 runs change one setting of the starting settings; NOISE = 0.0072.

Run	Setting tested	Seed	RMSE	Best epoch	Result
gru-01	starting settings (embedding 64, 64 units, dropout 0.2)	42	0.5844	2	start
gru-02	sequence length 24	42	0.5840	2	no gain
gru-03	sequence length 48	42	0.5854	2	no gain
gru-04	embedding 32	42	0.5887	2	no gain
gru-05	embedding 128	42	0.5861	2	no gain
gru-06	32 units	42	0.5866	2	no gain

Run	Setting tested	Seed	RMSE	Best epoch	Result
gru-07	128 units	42	0.5871	2	no gain
gru-08	dropout 0	42	0.5905	2	no gain
gru-09	dropout 0.4	42	0.5769	2	kept (final)
gru-10	learning rate 0.0003	42	0.5787	3	no gain
gru-11	learning rate 0.003	42	0.5839	1	no gain
gru-12	starting settings, new seed	43	0.5744	2	noise check
gru-13	starting settings, new seed	44	0.5813	2	noise check
gru-15	final settings (gru-09), new seed	43	0.5731	2	seed check
gru-16	final settings (gru-09), new seed	44	0.5785	2	seed check
gru-17	square-root class weights	42	0.5864	2	not kept
gru-18	square-root class weights	43	0.5936	2	not kept
gru-19	square-root class weights	44	0.6102	2	not kept
gru-20	balanced class weights	42	0.6294	2	not kept
gru-21	balanced class weights	43	0.6359	2	not kept
gru-22	balanced class weights	44	0.6689	3	not kept
gru-23	GloVe embedding, frozen	42	0.5758	18	not kept
gru-24	GloVe embedding, frozen	43	0.5720	22	not kept
gru-25	GloVe embedding, frozen	44	0.5796	20	not kept
gru-26	GloVe embedding, fine-tuned	42	0.5722	9	not kept
gru-27	GloVe embedding, fine-tuned	43	0.5689	8	not kept
gru-28	GloVe embedding, fine-tuned	44	0.5689	8	not kept